

Campus Marketplace

Sprint 1 — Task Assignment Plan

Software Design 2026 | 6 Developers | Firebase + Google Auth | Plain HTML/JS

1. Sprint 1 Overview

Sprint 1 establishes the foundation of the Campus Marketplace platform, focusing on two core epics: user identity and authentication (E1) and item listing management (E2). Completing these epics unlocks all buyer and transaction features planned for Sprint 2.

Sprint goal

Deliver a working registration, login, and role system backed by Firebase Auth and Google OAuth, together with full listing creation, editing, and type-flagging capability. All features must be deployed to Firebase Hosting via the CI/CD pipeline before sprint close.

Story summary

Dev	Story	Description	Epic	Priority	Points
Dev 1	—	Project setup & Firebase config	Infra	Blocker	~3
Dev 2	US1	Student registration & login	E1	High	5
Dev 3	US2	Role assignment	E1	High	3
Dev 4	US3	Create an item listing	E2	High	5
Dev 5	US4	Edit or remove a listing	E2	Medium	3
Dev 6	US5	Mark listing type	E2	Medium	2

Critical day-1 dependencies

- Dev 1 must share Firebase config before anyone else can connect to the project.
- Dev 3 must finalise the Firestore user schema before Dev 2 can store user profiles on first login.
- Dev 4 and Dev 6 must agree on the listing Firestore schema (especially the listingType field) before either writes to Firestore.
- Firebase Storage bucket permissions must be set up by Dev 1 before Dev 4 can test photo uploads end-to-end.
- Firestore security rules for listings (Dev 5) depend on the role schema defined by Dev 3.

Definition of done

- All acceptance tests pass with no failing scenarios.
- Code is reviewed and merged to main via pull request.
- Firestore security rules are deployed and tested in the Firebase console.
- UI components are responsive — tested on mobile and desktop viewports.
- CI/CD pipeline (GitHub Actions → Firebase Hosting) is active and passing.
- No critical or high-severity bugs remain open at sprint close.
- Each team member has regular, incremental commits spread across the sprint (not a last-minute spike).

2. Developer Task Breakdown

Dev 1 Dev 1 — Project setup & Firebase config

Priority: Blocker / Infra **Story Points:** ~3 pts **Sprint:** 1

This is the foundation task — no one else can connect to Firebase, run the app, or open pull requests until this is complete. Assign to the most organised team member. Must be done on day one.

Tasks

#	Task	Status
1	Create and configure the Firebase project (enable Auth, Firestore, Storage, Hosting)	To Do
2	Set up Git repo folder structure: /public, /src, /functions	To Do
3	Add Firebase SDK to the HTML shell and confirm connection works	To Do
4	Set up GitHub Actions CI/CD — lint + deploy to Firebase Hosting on push to main	To Do
5	Write .env.example and a README with setup instructions for all teammates	To Do
6	Share Firebase config keys with the team via a secure channel (never commit to repo)	To Do

Acceptance tests (Given / When / Then)

Test 1

Given a teammate clones the repo and follows the README

When they run the setup steps

Then the app loads and Firebase connects without errors

Test 2

Given a push is made to the main branch

When the GitHub Actions workflow triggers

Then the app builds and deploys to Firebase Hosting successfully

Dev 2 US1 — Student registration & login

Priority: High **Story Points:** 5 pts **Sprint:** 1

Implements Google OAuth via Firebase Auth. Every other feature depends on users being authenticated. Coordinate with Dev 3 on the Firestore user schema before writing the profile-storage task.

Tasks

#	Task	Status
1	Configure Google OAuth in the Firebase console (enable provider, set redirect URIs)	To Do
2	Build the login and registration page UI (login.html)	To Do
3	Integrate signInWithPopup / signInWithRedirect using the Firebase Auth SDK	To Do
4	On first login, write a user profile document to Firestore (uid, email, displayName, role: 'student')	To Do
5	Implement route guards — check onAuthStateChanged and redirect to login if unauthenticated	To Do
6	Write acceptance tests covering all three Given/When/Then scenarios below	To Do

Acceptance tests (Given / When / Then)

Test 1

Given a new student visits the site

When they click Sign Up and complete the Google OAuth flow

Then their account is created and they are redirected to the home page as a Student role

Test 2

Given a registered student is logged out

When they enter valid credentials

Then they are authenticated and land on their dashboard

Test 3

Given an unauthenticated user tries to access a protected page

When they navigate directly via URL

Then they are redirected to the login page

Dev 3 US2 — Role assignment

Priority: High **Story Points:** 3 pts **Sprint:** 1

Defines who can do what across the entire platform. Firestore schema and RBAC must be finalised before Dev 2 can store user profiles and before Dev 5 can write listing security rules. Share the agreed schema in the team Slack/group chat immediately.

Tasks

#	Task	Status
1	Define and document the Firestore user schema (uid, email, role, createdAt)	To Do
2	Implement RBAC using Firebase custom claims via a Cloud Function or Admin SDK script	To Do
3	Build the admin panel page — list all users, assign or change role, save to Firestore	To Do
4	Write Firestore security rules blocking students from /admin paths	To Do
5	Write tests confirming that a student receives an Access Denied response on the admin URL	To Do

Acceptance tests (Given / When / Then)

Test 1

Given a logged-in admin opens the user management panel

When they assign a user the Trade Facility Staff role and save

Then that user gains staff feature access and loses student-only access

Test 2

Given a student user is authenticated

When they attempt to access the admin dashboard URL directly

Then they receive an Access Denied response

Dev 4 US3 — Create an item listing

Priority: High **Story Points:** 5 pts **Sprint:** 1

The core seller experience. This story produces the data that every buyer feature in Sprint 2 depends on. Coordinate with Dev 6 on the listing schema (especially the listingType field) before writing any Firestore documents.

Tasks

#	Task	Status
1	Design and build the listing creation form UI (create-listing.html)	To Do
2	Implement photo upload (min 1, max 5 photos) to Firebase Storage — store download URLs in Firestore	To Do
3	Build the category dropdown (Textbooks, Electronics, Furniture, Clothing, Other)	To Do
4	Build the condition selector (New, Like New, Good, Fair, Poor)	To Do
5	Add the listing type toggle: For Sale / For Trade / Either (coordinate field name with Dev 6)	To Do

#	Task	Status
6	Save the listing document to Firestore with sellerUID, timestamp, and all form fields	To Do
7	Show a success confirmation message and redirect to the new listing detail page	To Do
8	Implement client-side validation — show inline errors for empty required fields	To Do

Acceptance tests (Given / When / Then)

Test 1

Given a logged-in student completes all required listing fields

When they submit the form

Then a new listing is created and visible on the marketplace

Test 2

Given a student creates a listing and uploads a photo

When the form is submitted

Then the photo is stored in Firebase Storage and displayed on the listing page

Test 3

Given a student submitting a listing leaves a required field empty

When they click Submit

Then the form shows a validation error and does not submit

Dev 5 US4 — Edit or remove a listing

Priority: Medium **Story Points:** 3 pts **Sprint:** 1

Protects listing ownership on the frontend and in Firestore security rules. Depends on Dev 4's listing schema and Dev 3's role schema being in Firestore first. Start this story once Dev 3 has published the user schema.

Tasks

#	Task	Status
1	Show Edit and Delete buttons only when the logged-in user is the listing owner (match uid)	To Do
2	Build a pre-populated edit form — load existing listing data from Firestore on page load	To Do
3	Implement delete with a confirmation dialog before issuing the Firestore delete call	To Do
4	Write Firestore security rules: only the owner (request.auth.uid == resource.data.sellerUID) may update or delete a listing	To Do
5	Test all three acceptance scenarios, including direct URL access attempted by a non-owner	To Do

Acceptance tests (Given / When / Then)

Test 1

Given a logged-in seller views their own listing

When they click Edit, update the price, and save

Then the listing reflects the new price immediately

Test 2

Given a logged-in seller views their own listing

When they click Delete and confirm

Then the listing is removed from the marketplace

Test 3

Given a student who did not create a listing

When they attempt to access the edit URL directly

Then they are denied access with an error

Dev 6 US5 — Mark listing type

Priority: Medium **Story Points:** 2 pts **Sprint:** 1

Adds the For Sale / For Trade / Either distinction to listings and the browse page filter. This story has the fewest points but requires close coordination with Dev 4 — agree on the listingType field name and values on day one, then Dev 6 can add the toggle as a PR on top of Dev 4's form.

Tasks

#	Task	Status
1	Agree listingType field name and allowed values with Dev 4 on day one (e.g. 'sale', 'trade', 'either')	To Do
2	Add the listing type toggle UI to the creation form (coordinate with Dev 4 — add as a separate PR)	To Do
3	Display a For Sale / For Trade / Either badge prominently on the listing card and detail page	To Do
4	Build the browse/marketplace page (browse.html) with a filter by listing type	To Do
5	Ensure listings marked Either appear in both For Sale and For Trade filter results	To Do
6	Write acceptance tests for badge display and filter behaviour	To Do

Acceptance tests (Given / When / Then)

Test 1

Given a seller creates a listing and selects For Trade

When the listing is published

Then a For Trade badge is visible on the listing card and detail page

Test 2

Given a buyer browses listings and filters by For Sale

When the filter is applied

Then only listings marked For Sale or Either are shown

3. Academic Integrity Reminders

The following requirements apply to every team member for every sprint, as stated in the project brief.

Git commit history

- Commits must be incremental and spread across the sprint — not clustered in the 24 hours before the deadline.
- Each commit must be attributable to the individual who wrote the code (commit under your own GitHub account).
- Markers will review the full git commit history. A large spike of commits immediately before a deadline is a red flag.

CI/CD pipeline

- The GitHub Actions pipeline must be active and building throughout the sprint — not set up at the last minute.
- Build and test history should reflect ongoing development.

Sprint retrospective (individual submission)

- Each team member must individually submit a written retrospective (200–400 words) at sprint end.
- It must describe what you personally built, what you found difficult, what you would do differently, and how your contribution fits the overall system.
- Retrospectives will be cross-referenced with your git commit history. Claiming credit for work not reflected in your commits is treated as academic dishonesty.

Sprint review viva

- At the sprint assessment, the marker will ask individual team members to explain specific parts of the codebase — walking through a function, justifying a design decision, or modifying a value live.
- Members who cannot explain the code they are presenting will receive zero for the Viva component regardless of overall submission quality.